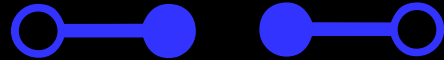


Dezvoltarea aplicațiilor Web



managementul datelor RDF

Prof. Sabin Corneliu Buraga
profs.info.uaic.ro/sabin.buraga/

“Trebuie să adunăm fapte
pentru a avea idei.”

Georges-Louis de Buffon



Ce modele de date sunt disponibile pe Web?

modele de date disponibile pe Web

textuale

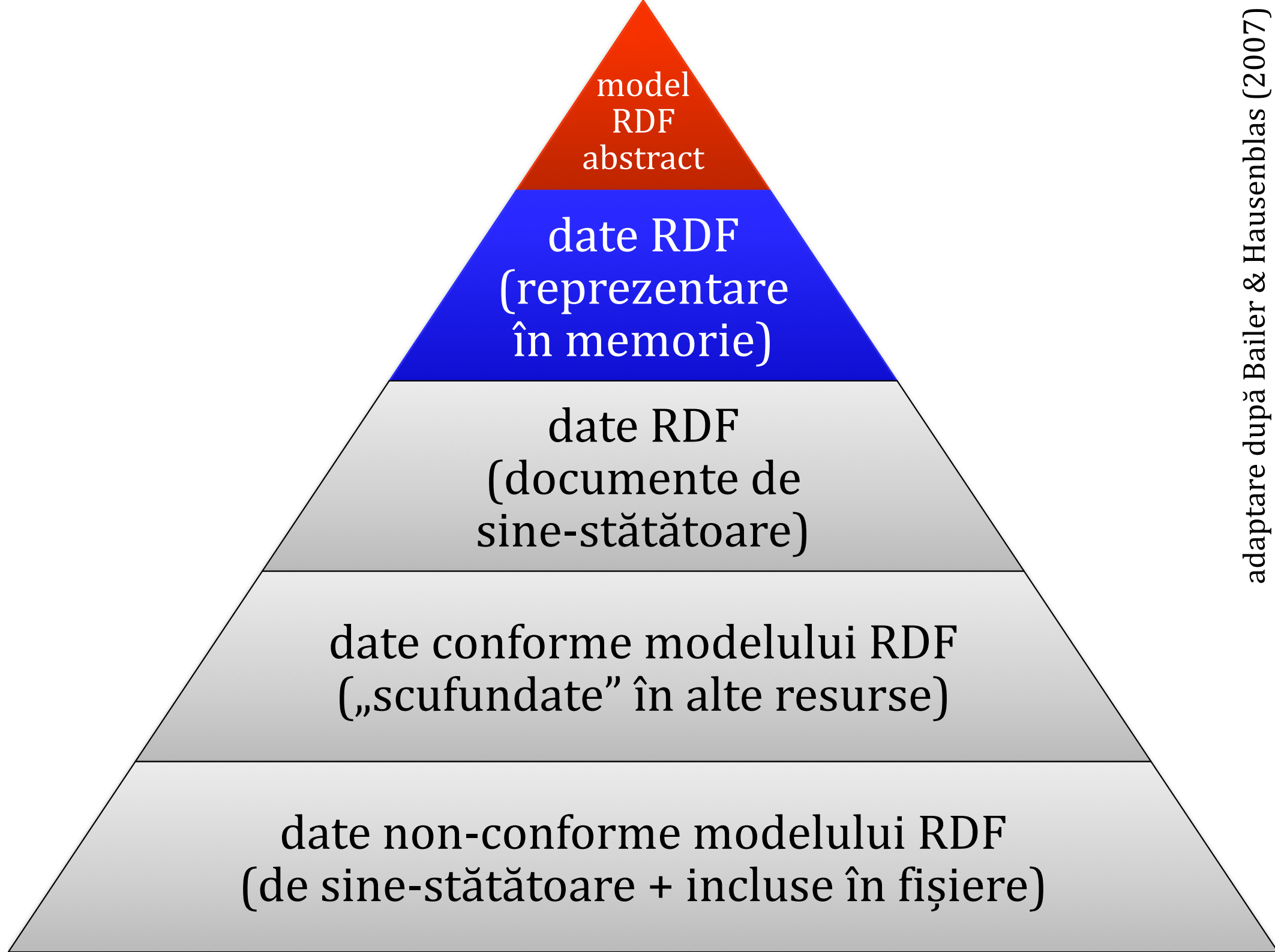
date nestructurate – eventual, multilingve

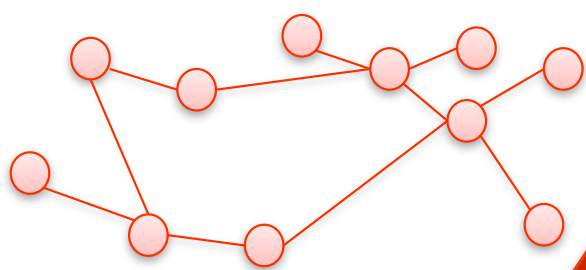
structurate

RDF – reprezintă „lucruri” (resurse) din lumea reală: indivizi, servicii, aplicații,... – de dorit, modelate formal

hibride

date structurate „scufundate” în text
e.g., microdate HTML, **RDFa**, **JSON-LD**





model
RDF
abstract

date RDF
(reprezentare
în memorie)

cod binar

RDF/XML, N3, Turtle,
JSON-LD,... (serializări)

date RDF
(documente de
sine-stătătoare)

date conforme modelului RDF
(„scufundate” în alte resurse)

RDFa

baze de date
relaționale, XML,
HTML, microdate
schema.org, JSON,...

non-conforme modelului RDF
(sine-stătătoare + incluse în fișiere)



Ce instrumente de procesare
a declarațiilor RDF putem folosi?

necesități

Procesor (*parser*) RDF

preluarea datelor disponibile în diverse formate
și reprezentarea lor ca triple RDF

api-uri & biblioteci rdf – **exemple**

Apache Jena (Java) – jena.apache.org

ARC2 (PHP) – github.com/semsol/arc2

dotNetRDF (C#) – www.dotnetrdf.org

EasyRDF (PHP) – www.easyrdf.org

RDF.js (JavaScript) – rdf.js.org

RDF.rb (Ruby) – ruby-rdf.github.io

RDFLib (Python) – rdflib.readthedocs.io

Redland (C; portări în PHP, Python, Ruby,...) – librdf.org

alte la www.w3.org/2001/sw/wiki/Tools

necesități

Serializator (*serializer*) RDF

exprimarea modelului intern RDF
în diverse formate

N-Triples, RDF/XML, Turtle (N3), TriX, RDFa, JSON-LD,...

(de)compresie triple RDF
în formatul binar **HDT** (*Header, Dictionary, Triples*)

necesități

Convertor RDF

preluarea datelor din alte modele
și transformarea lor în RDF

extragerea directă din documentele HTML
via *scrapers/harvesters*

și/sau

instrumente bazate pe LLM (*Large Language Model*)

instrumente

Extractoare/convertoare RDF – exemple:

EasyRDF (PHP) – www.easyrdf.org/converter

HDT-it! (C++, Java) – www.rdfhdt.org

Raptor (C) – librdf.org/raptor/

RDF Converter (JS) – github.com/zazuko/rdf-converter

Tarql (*CSV to RDF using SPARQL*) – tarql.github.io

RDF Scholar – RDF Conversion and Integration
www.yeschat.ai/gpts-ZxX7cnfd-RDF-Scholar

necesități

Navigatoare RDF (*hyperdata browsers*)

redau utilizatorului (uman) o reprezentare „frumoasă” a triplelor RDF, permitând interacțiunea cu datele: vizualizare, filtrare,...

necesități

Extragerea și procesarea datelor existente pe Web

o soluție „clasică”: *information extraction*
(eventual, pe baza procesării limbajului natural – NLP)
e.g., a se experimenta **OpenRefine** – openrefine.org

în cazul nostru, dorim să recurgem la
tehnologiile Web-ului semantic

necesități

O modalitate standardizată de „convertire”
în triple RDF
a construcțiilor semantice încapsulate
în documentele Web

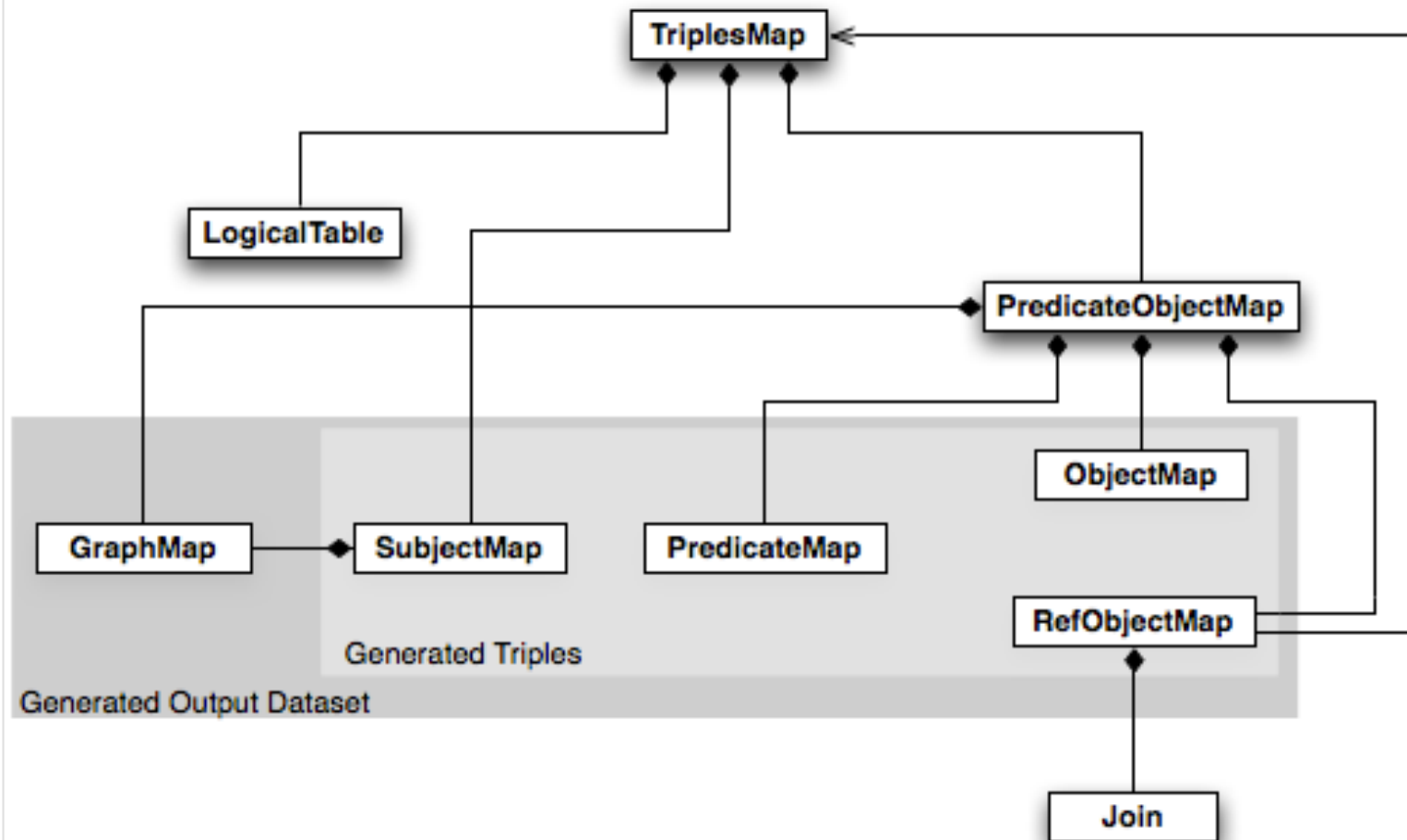
necesități

Date stocate în sisteme de baze de date relaționale



date modelate cu RDF

asocieri directe – *mappings* (via schema bazei de date)



realizarea diferitelor asocieri exprimate via
R2RML – RDB to RDF Mapping Language
recomandare a Consorțiului Web (2012)
www.w3.org/TR/r2rml/

baze de date relaționale gestionate via SQL

```
CREATE TABLE "Software" (  
    "ID" INT, PRIMARY KEY("ID"), "nume" CHAR(69), "desc" CHAR(256)  
)  
CREATE TABLE "Utiliz" (  
    "ID" INT, PRIMARY KEY("ID"), "identit" CHAR(101),  
    "soft" INT, FOREIGN KEY("soft") REFERENCES "Software"("ID")  
)  
INSERT INTO "Software" ("ID", "nume", "desc") VALUES (13, 'Fedora', '...')  
INSERT INTO "Utiliz" ("ID", "identit", "soft") VALUES (7, 'Andrei', 13)  
INSERT INTO "Utiliz" ("ID", "identit", "soft") VALUES (8, 'Maria', NULL)
```

baze de date relaționale gestionate via SQL

```
CREATE TABLE "Software" (  
  "ID" INT, PRIMARY KEY("ID"), "nume" CHAR(69), "desc" CHAR(256)  
)  
CREATE TABLE "Utiliz" (  
  "ID" INT, PRIMARY KEY("ID"), "identit" CHAR(101),  
  "soft" INT, FOREIGN KEY("soft") REFERENCES "Software"("ID")  
)  
INSERT INTO "Software" ("ID", "nume", "desc") VALUES (13, 'Fedora', '...')  
INSERT INTO "Utiliz" ("ID", "identit", "soft") VALUES (7, 'Andrei', 13)  
INSERT INTO "Utiliz" ("ID", "identit", "soft") VALUES (8, 'Maria', NULL)
```

date RDF

```
<Utiliz/ID=7> rdf:type          <Utiliz> .  
<Utiliz/ID=7> <Utiliz#ID>      7 .  
<Utiliz/ID=7> <Utiliz#identit> "Andrei" .  
<Utiliz/ID=7> <Utiliz#soft>    13 .  
<Utiliz/ID=7> <Utiliz#ref-software> <Software/ID=13> .  
...  
<Software/ID=13> rdf:type          <Software> .  
<Software/ID=13> <Software#ID>    13 .  
<Software/ID=13> <Software#nume>  "Fedora" .  
<Software/ID=13> <Software#desc>  "An Open Linked Data Platform by LYRASIS" .
```

asocieri directe

recomandare W3C
(M. Arenas *et al.*, 2012)

[www.w3.org/TR/
rdb-direct-mapping/](http://www.w3.org/TR/rdb-direct-mapping/)



Nu există posibilitatea utilizării unui sistem de stocare/interogare a triplelor RDF?

sisteme de stocare rdf

Datele RDF sunt menținute persistente
în cadrul unui *RDF store*

sisteme de stocare rdf

Oferă posibilitatea fuzionării (*merging*)
seturilor de date RDF

date fiind 2 seturi de triple RDF, în urma fuzionării
va rezulta un set de triple RDF ce include toate triplele
din ambele seturi de intrare în care resursele
având același URI sunt considerate echivalente

sisteme de stocare rdf

Intern, un *RDF store* poate stoca triplele
într-o varietate de formate

baze de date (non-)relaționale

versus

structuri de date speciale – eventual, în memorie

inclusiv, la nivel de *cloud*

sisteme de stocare rdf

Intern, un *RDF store* poate stoca triplele într-o varietate de formate

dataset $\equiv$ colecție de grafuri RDF:
triple RDF

+

context desemnat de un URI (*named graph*)

sisteme de stocare rdf

Intern, un *RDF store* poate stoca triplele într-o varietate de formate

dataset

<subject> <predicate> <object> <graph label>

`_:anca <http://schema.org/relatedTo> _:eugen <http://social.void/graphs/> .`

format standardizat: N-Quads (2014)

www.w3.org/TR/n-quads/

sisteme de stocare rdf

Asigurarea interoperabilității este facilitată de formatele de serializare standardizate

RDF/XML, Turtle, JSON(-LD)

sisteme de stocare rdf

Aspecte de interes:

disponibilitatea

performanța – *e.g.*, scalabilitatea, timpul de răspuns,...

capacitatea de stocare

facilitățile privind interogarea datelor (inclusiv tranzacții)

API-urile disponibile

fiabilitatea

securitatea

efectuarea de raționamente automate

sisteme de stocare rdf

Exemple de implementări curente
(unele disponibile în regim *open source* ori *freeware*)

stocare în structuri de date native:

AllegroGraph (REST API + Java, Lisp, Python SDKs)

AnzoGraph DB (REST API, C++, Java)

Apache Jena TDB (Java) – jena.apache.org

Eclipse RDF4J (Java) – rdf4j.org

Fluree (Closure) – developers.flur.ee

Oracle RDF Graph Server & Query UI

Stardog (REST API, Java, .NET) – docs.stardog.com

sisteme de stocare rdf

Exemple de implementări curente
(unele disponibile în regim *open source* ori *freeware*)

folosind sisteme tradiționale de baze de date:

OpenLink Virtuoso (C, C#, Java, JS, PHP, Ruby, XQuery,...)

OntoText GraphDB (Java)

RDFFox (REST API + Java) – docs.oxfordsemantic.tech

sisteme de stocare rdf

Exemple de implementări curente
(unele disponibile în regim *open source* ori *freeware*)

bazate pe sisteme NoSQL:

Amazon Neptune (AWS, *serverless*), BrightstarDB (.NET),
RDF Graph for Oracle NoSQL DB

recurgând la baze de date native XML: MarkLogic
(interogări XQuery, SQL, SPARQL + API-uri diverse)
developer.marklogic.com/learn/semantics-exercises

sisteme de stocare rdf

Suport pentru realizarea de inferențe (simple)

Jena – specificarea regulilor procesate de un *rule engine*
+ API pentru realizarea raționamentelor (*reasoning*)

Eclipse RDF4J – raționamente privitoare la RDF Schema

MarkLogic – raționamente vizând tipul resurselor și
relațiile dintre ele

RDFOX – raționamente bazate pe reguli +
explicarea deducțiilor realizate automat

Stardog – raționamente automate
prin expandarea interogărilor (*query expansion*)

sisteme de stocare rdf: utilizări

Ontotext GraphDB folosit de BBC, Springer Nature,
LinkedLifeData și altele;

inclus în GATE (*General Architecture for Text Engineering*)

Best Buy, Ericsson, NASA și Salesforce recurg la **Stardog**

OpenLink Virtuoso utilizat și de proiectele DBpedia,
Linked Open Data Cloud și UniProt

data.gov.uk se bazează pe **Apache Jena TDB**

MarkLogic folosit în cadrul Airbus, Chevron, EATON,...

platforma Legislate gestionează resursele cu **RDFFox**

Fluree: *resolving beef traceability problem with RDF & blockchain*

sisteme de stocare rdf

Suport pentru constrângeri impuse datelor

cadrul general: SHACL – *Shapes Constraint Language*
suită de recomandări ale Consorțiului Web (2017)

www.w3.org/TR/shacl/

validarea grafurilor RDF pe baza unor condiții (*shapes*)
asupra drumurilor (*paths*) dintre noduri

exemplul #1

@prefix schema: <http://schema.org/> .
@prefix sh: <<http://www.w3.org/ns/shacl#>> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

schema:PersonShape

```
a sh:NodeShape ;
sh:targetClass schema:Person ;
sh:property [
  sh:path schema:givenName ;
  sh:datatype xsd:string ;
  sh:minLength "33"^^xsd:byte
];
sh:property [
  sh:path schema:birthDate ;
  sh:maxCount "1"^^xsd:byte ;
  sh:lessThan schema:deathDate
];
sh:property [
  sh:path schema:gender ;
  sh:in ( "female" "male" );
].
```

```
:Tuxy a schema:Person ;
  schema:givenName true ;
  schema:givenName "Tuxy" ;
  schema:gender "n/a" ;
  schema:birthDate "2025-12-31"^^xsd:date ;
  schema:birthDate "1974-01-07"^^xsd:date ;
  schema:deathDate "2025-12-30"^^xsd:date .
```

graful datelor
de validat

@prefix schema: <http://schema.org/> .
@prefix sh: <<http://www.w3.org/ns/shacl#>> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

schema:PersonShape

```
a sh:NodeShape ;
sh:targetClass schema:Person ;
sh:property [
  sh:path schema:givenName ;
  sh:datatype xsd:string ;
  sh:minLength "33"^^xsd:byte
];
sh:property [
  sh:path schema:birthDate ;
  sh:maxCount "1"^^xsd:byte ;
  sh:lessThan schema:deathDate
];
sh:property [
  sh:path schema:gender ;
  sh:in ( "female" "male" );
].
```

constrângeri impuse
entităților din categoria
schema.org/Person

numele propriu e un șir
de minim 33 caractere,
data nașterii trebuie
să fie unică + să aibă
valoare mai mică decât
cea a morții, iar genul
trebuie să ia o valoare
dintr-o listă

```
:Tuxy a schema:Person ;
schema:givenName true ;
schema:givenName "Tuxy" ;
schema:gender "n/a" ;
schema:birthDate "2025-12-31"^^xsd:date ;
schema:birthDate "1974-01-07"^^xsd:date ;
schema:deathDate "2025-12-30"^^xsd:date .
```

graful datelor
de validat

validateDatatype(true, xsd:string) ▶ 1 violation

validateDatatype(Tuxy, xsd:string)

validateMinLength(true, 33) ▶ 1 violation

validateMinLength(Tuxy, 33) ▶ 1 violation

validateMaxCountProperty(s:Tuxy, schema:birthDate, 1) ▶ 1 violation

validateLessThanProperty(s:Tuxy, schema:birthDate, schema:deathDate) ▶ 1 violation

validateIn(n/a, Blank node _:n1525) ▶ 1 violation

raport disponibil
în format RDF

```
[  
  a sh:ValidationResult ;  
  sh:resultSeverity sh:Violation ;  
  sh:sourceConstraintComponent sh:InConstraintComponent ;  
  sh:sourceShape _:n1523 ;  
  sh:focusNode <http://www.infoiasi.ro/wade#Tuxy> ;  
  sh:value "n/a" ;  
  sh:resultPath schema:gender ;  
  sh:resultMessage "Value is not in Blank node _:n1525" ;  
]
```

Shapes Graph Structure

- Shapes with Target (1)
 - schema:PersonShape
 - Property constraint: Blank node _:n1522
 - Property constraint: Blank node _:n1523
 - on property path: schema:gender
 - In constraint
 - has component: sh:InConstraintComponent
 - in: Blank node _:n1525
 - Target Nodes (1)
 - s:Tuxy
- Constraint Components (38)

exemplul #2

```
# restricții impuse datelor vizând o instanță
# schema.org/MobileApplication
schema:MobileAppShape a sh:NodeShape ;
  sh:targetClass schema:MobileApplication ;
  sh:property [
    sh:path schema:name ;
    sh:datatype xsd:string
  ],
[ # data publicării e opțională
  sh:path schema:datePublished ;
  sh:maxCount 1 ; sh:minCount 0 ;
  sh:datatype xsd:date
],
[ sh:path schema:applicationCategory ;
  sh:minCount 1 ;
  # categoria e un IRI/literal
  sh:nodeKind sh:IRIOrLiteral
],
[ # descrierea trebuie să includă
  # sufixul limbii
  sh:path schema:description ;
  sh:datatype rdf:langString
]
.
```

```
{
  "@context": {
    "sh": "http://www.w3.org/ns/shacl#",
    "schema": "http://schema.org/"
  },
  "@graph": [
    {
      "@id": "_:g0",
      "sh:datatype": {
        "@id": "http://www.w3.org/2001/XMLSchema#string"
      },
      "sh:path": {
        "@id": "schema:name"
      }
    },
    {
      "@id": "_:g1",
      "sh:datatype": {
        "@id": "http://www.w3.org/2001/XMLSchema#date"
      },
      "sh:maxCount": {
        "@type": "http://www.w3.org/2001/XMLSchema#integer",
        "@value": "1"
      },
      "sh:minCount": {
        "@type": "http://www.w3.org/2001/XMLSchema#integer",
        "@value": "0"
      }
    }
  ]
}
```

restricțiile
exprimate
în JSON-LD

:angry-profs a

schema:MobileApplication ;

schema:name "Angry Profs" ;

corect

schema:applicationCategory

"http://dbpedia.org/resource/Electronic_game" ;

corect: obiectul reprezintă o adresă Web

schema:applicationCategory

"http://dbpedia.org/resource/Electronic_game"^^sh:IRI ;

corect: valoarea obiectului e un literal (număr)

schema:applicationCategory 33 ;

incorrect

schema:datePublished "null" ;

corect

schema:datePublished "2022-11-01"^^xsd:date ;

incorrect: nu se specifică limba

schema:description "...";

corect

schema:description "...">@ro .

date RDF
de validat

alte detalii în cartea disponibilă liber: J. Gayo *et al.*,
Validating RDF Data, Morgan & Claypool (2018)

www.validatingrdf.com

Shapes Graph

```
7 schema:MobileAppShape a sh:NodeShape ;
8 sh:targetClass schema:MobileApplication ;
9 sh:property [
10   sh:path schema:name ;
11   sh:datatype xsd:string
12 ],
13 [
14   # data publicarii e optionala
15   sh:path schema:datePublished ;
16   sh:maxCount 1 ;
17   sh:minCount 0 ;
18   sh:datatype xsd:date
19 ],
20 [
21   # categoria e indicata via o adresa Web sau un literal
22   sh:path schema:applicationCategory ;
23   sh:minCount 1 ;
24   sh:nodeKind sh:IRIOrLiteral
```

Data Graph

```
1 @prefix schema: <http://schema.org/> .
2 @prefix sh: <http://www.w3.org/ns/shacl#> .
3 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
4
5 _:angry-profs a schema:MobileApplication ;
6   schema:name "Angry Profs" ;
7   # corect: valoarea obiectului e un sir de caractere
8   schema:applicationCategory
9     "http://dbpedia.org/resource/Electronic_game" ;
10  # corect: obiectul reprezinta o adresa Web
11  schema:applicationCategory
12    |"http://dbpedia.org/resource/Electronic_game"^^sh:IRI
13  # corect: valoarea obiectului e un literal (numar)
14  schema:applicationCategory 33 ;
15  # incorect: tip incorect de date
16  schema:datePublished "null" ;
17  # corect
18  schema:datePublished "2022-11-01"^^xsd:date ;
```

Success

No

Errors found

- angry-profs:
 - schema:datePublished:
 - Value does not have datatype <http://www.w3.org/2001/XMLSchema#date>
 - More than 1 values
 - schema:description:
 - Value does not have datatype <http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>

vezi arhiva
cu exemplificări

de experimentat instrumentele Web

SHACL Playgrounds

shacl-playground.zazuko.com • shacl-play.sparna.fr/play/



Există un limbaj de interogare
a seturilor de triple RDF?

sparql

SPARQL Protocol and RDF Query Language

limbaj descriptiv standardizat
pentru realizarea de interogări pe baza potrivirilor
de șabloane referitoare la triple RDF

sparql

Oferă suport pentru manipularea construcțiilor RDF

Pune la dispoziție operații asupra grafurilor RDF

Independent de platforma software utilizată

limbaj de interogare + protocol de acces la triple RDF

sparql: limbaj de interogare

Recomandări ale Consorțiului Web:

SPARQL 1.0 (2008)

permite doar acces la date (interogări *read-only*)

www.w3.org/TR/rdf-sparql-query/

SPARQL 1.1 (2013)

oferă și posibilități de actualizare a datelor

www.w3.org/TR/sparql11-overview/

SPARQL 1.2 (în lucru: 06 noiembrie 2025)

www.w3.org/TR/sparql12-query/

sparql: limbaj de interogare

Modelul de interogare e fundamentat de
șabloane de grafuri RDF – *graph patterns*

B. DuCharme, *Learning SPARQL* (2nd Ed.), O'Reilly (2013)

www.learningsparql.com

www.bobdc.com/categories/sparql/

sparql: limbaj de interogare

Termeni RDF (RDF-T):

reuniunea mulțimilor IRI, noduri blanc, literal RDF

Variabile de interogare (V):

simboluri prefixate de „?” sau de „\$”

Șablon (*triple pattern*): membru al mulțimii
 $(\text{RDF-T} \cup V) \times (\text{IRI} \cup V) \times (\text{RDF-T} \cup V)$

Șabloane de tip graf: mulțimi de șabloane

sparql: limbaj de interogare

Convenții sintactice bazate pe Turtle/N3

triplele RDF sunt considerate
ca fiind tablouri de termeni RDF

termen RDF = IRI | nod blanc | literal

pattern de triple = tablou de termeni RDF
sau nume de variabile (termeni liberi)

sparql: limbaj de interogare

URI – sau, mai general, IRI – delimitați de „<” și „>”

situl Web al unei persoane:

[<https://profs.info.uaic.ro/sabin.buraga/>](https://profs.info.uaic.ro/sabin.buraga/)

o proprietate definită de un vocabular:

[<http://xmlns.com/foaf/0.1/name>](http://xmlns.com/foaf/0.1/name)

[<http://schema.org/servesCuisine>](http://schema.org/servesCuisine)

o clasă de resurse (*i.e.* un concept):

[<http://dbpedia.org/ontology/Film>](http://dbpedia.org/ontology/Film)

[<http://schema.org/Winery>](http://schema.org/Winery)

sparql: limbaj de interogare

Pentru a referi spații de nume ale vocabularelor utilizate se folosește construcția **PREFIX**

PREFIX dc:	<code><http://purl.org/dc/elements/1.1/></code>
PREFIX foaf:	<code><http://xmlns.com/foaf/0.1/></code>
PREFIX dbo:	<code><http://dbpedia.org/ontology/></code>
PREFIX dbp:	<code><http://dbpedia.org/property/></code>
PREFIX wd:	<code><http://www.wikidata.org/entity/></code>
PREFIX geo:	<code><http://www.w3.org/2003/01/geo/wgs84_pos#></code>
PREFIX yago:	<code><http://yago-knowledge.org/resource/></code>

lista prefixelor folosite uzual: prefix.cc

sparql: limbaj de interogare

Nodurile blanc se specifică prin **_:nume**
sau sunt delimitate de **[și]**
dacă se folosesc o singură dată

sparql: limbaj de interogare

Literali RDF

(șiruri de caractere cu attribute opționale)

"Web"

"Web"@ro

se indică limba

"true"^^xsd:boolean

se specifică explicit...

"true"^^xsd:string

...tipul de date

sparql: limbaj de interogare

Abrevieri referitoare la triple

:s	:p	:o1 ,		:s	:p	:o1 .
		:o2 .	echivalent cu	:s	:p	:o2 .
:s	:p1	:o1 ;		:s	:p1	:o1 .
	:p2	:o2 .	echivalent cu	:s	:p2	:o2 .

sparql: limbaj de interogare

Abrevieri

privitoare la nodurile blanc

:s **:p** [**:p'** **:o**]

echivalent cu

:s **:p** **_****:x** . **_****:x** **:p'** **:o** .

sparql: limbaj de interogare

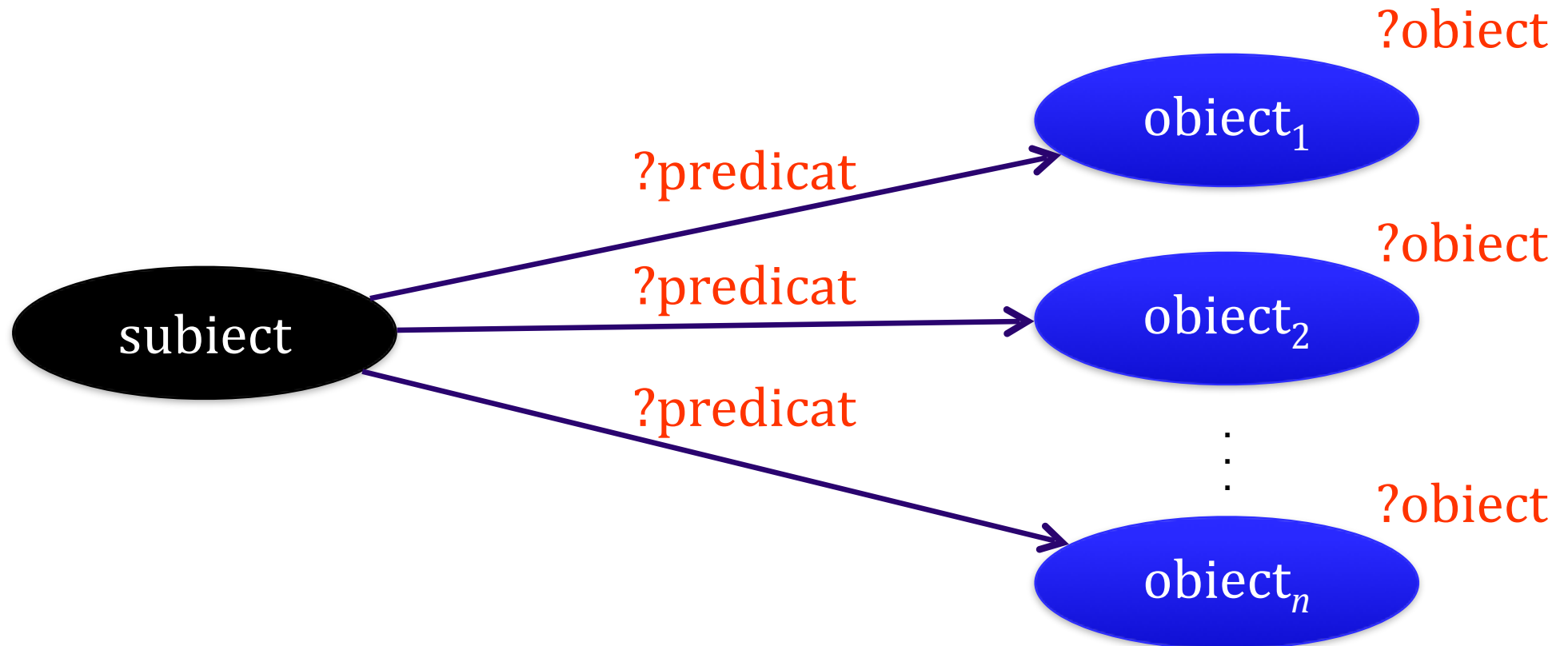
Un șablon (*pattern*) permite specificarea de variabile indicând date ce vor fi returnate în urma realizării interogării

?subiect ?predicat ?obiect .

sparql: limbaj de interogare

Șablon: **subiect** ?**predicat** ?**obiect** .

?**predicat** și ?**obiect** reprezintă necunoscutele (*unbounded*)



sparql: limbaj de interogare

Exemple:

descoperirea proprietăților/relațiilor
dintre subiect și obiect

_:subiect ?predicat _:obiect .

sparql: limbaj de interogare

Exemplificări:

lista persoanelor inter-conectate
via predicatul (proprietatea) *foaf:knows*
?persoana foaf:knows ?altaPersoana .

perechi locuri—coordonate geografice
folosind proprietatea specificată de schema.org
?loc schema:geo ?coordGeo .

sparql: limbaj de interogare

Șabloanele pot fi combinate pentru a forma șabloane mai complexe

- șablon de tip graf (BGP – *basic graph pattern*)

componente ale unei interogări SPARQL
exprimate formal – algebric – prin tuplul (E, DS, QF)

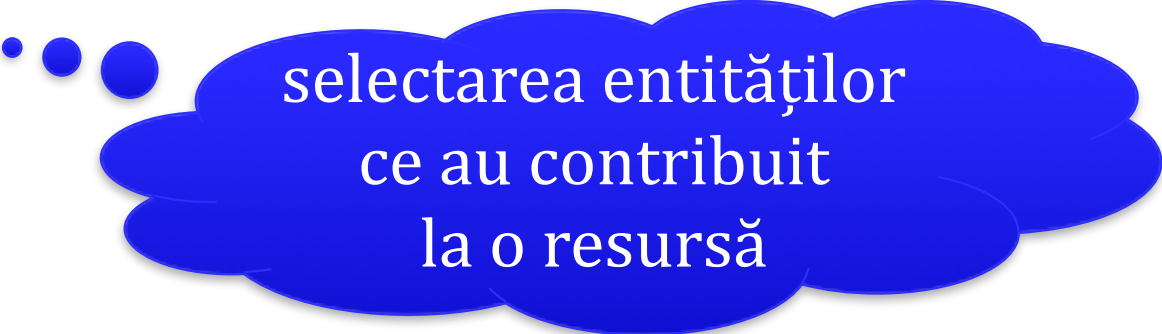
E = expresie SPARQL

DS = set de date RDF (*dataset*) local/disponibil pe Web

QF = formular de interogare (*query form*)

sparql: limbaj de interogare

```
SELECT ?contrib
WHERE {
  <http://en.wikipedia.org/>
  <http://purl.org/dc/elements/1.1/contributor>
  ?contrib .
}
```



selectarea entităților
ce au contribuit
la o resursă

{ ... } desemnează un *graph pattern*
în acest exemplu, se utilizează un predicat din vocabularul DCMI

sparql: limbaj de interogare

Rezultatul interogării poate fi:

un tabel de valori corespunzătoare variabilelor
utilizate, rezultat precizat prin **SELECT**

sparql: limbaj de interogare

Rezultatul interogării poate fi:

o valoare booleană întoarsă de construcția **ASK**

„este adevărat că...?”

sparql: limbaj de interogare

Rezultatul interogării poate fi:

un graf RDF creat via **CONSTRUCT** sau **DESCRIBE**

CONSTRUCT oferă un graf în care variabilele
sunt înlocuite cu valorile găsite

DESCRIBE întoarce un graf care descrie resursele găsite
(structura datelor e determinată de procesorul RDF)

sparql: limbaj de interogare

SELECT

rezultatul interogării reprezintă un tabel

secvență de rânduri compuse
din valorile variabilelor (*variable bindings*)



Studiu de caz: date (triple) RDF privitoare la studenți

```
s:ana    rdf:type foaf:Person ; foaf:firstName "Ana"@ro ;  
         foaf:gender "female" ; foaf:age 22 ;  
         foaf:knows s:bogdan, s:cristina, s:diana ;  
         foaf:title "Bachelor of Science"@en ; rel:closeFriendOf s:cristina .  
s:bogdan rdf:type foaf:Person ; ...  
         rel:knowsByReputation <http://www.w3.org/People/Berners-Lee/card#i> .  
s:cristina rdf:type foaf:Person , ac:Research_Student ; ... .  
s:diana   rdf:type foaf:Person ; ... rel:siblingOf s:cristina ; rel:worksWith s:ana .  
s:busaco  rdf:type foaf:Person , ac:Professor , ac:Student_Adviser ; ...  
         rel:influencedBy <http://www.w3.org/People/Berners-Lee/card#i> ;  
         rel:wouldLikeToKnow <http://www.w3.org/People/Berners-Lee/card#i> .  
s:dlucanu rdf:type ac:Professor , ac:Head_of_Department , ac:Student_Adviser ;  
         ... rel:collaboratesWith s:busaco .  
s:examineaza rdf:type rdf:Property ;  
             rdfs:domain foaf:Person ; rdfs:range foaf:Person .  
  
s:busaco      s:examineaza    s:ana , s:bogdan , s:cristina , s:diana .  
s:dlucanu     s:examineaza    s:bogdan , s:diana .
```

folosim vocabularele (modelele conceptuale) publice: RDF, RDF Schema, FOAF, *relationship*, AIISO (*Academic Institution Internal Structure Ontology Roles*)

Dataset Creation
Use this section of the GUI to import RDF data from Files/URIs. You can reset the Dataset to the empty dataset by clicking the Clear button

Import RDF from File:

Import RDF from URI:

Query

```

PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?persoana
WHERE {
  ?persoana a foaf:Person
}

```

SPARQL GUI - SPARQL Results Viewer - 5 ...

Format Values as

persoana
http://www.infoiasi.ro/wade/ana
http://www.infoiasi.ro/wade/bogdan
http://www.infoiasi.ro/wade/cristina
http://www.infoiasi.ro/wade/diana
http://www.infoiasi.ro/wade/busaco

Syntax Options

☒ SPARQL 1.0
☐ SPARQL 1.1
☐ SPARQL 1.1 Extended

SPARQL Results Format

Graph Format

Query Options

Timeout:

☒ Partial Results on Timeout ☒ Default Graph is union of all graphs
☒ Allow Graphs to be loaded 'on demand' from the Web? ☒ View Results and Graphs in Application
☐ Permit Unknown Function URIs ☒ Log Query Explanations
☒ Optimise Queries ☐ Enable Full Text Indexing
☒ Optimise Query Algebra ☐ Allow Unsafe Optimisations
☒ Enable Parallel Query Evaluation ☐ Use BOM for UTF-8 Output

Query Tools

Manage Queries:

Explain Queries:

Query Log:

determinarea resurselor de tip persoană

```

SELECT ?persoana
WHERE {
  ?persoana rdf:type foaf:Person
}

```

vezi arhiva cu
exemple asociate
cursului

sparql: limbaj de interogare

SELECT

oferirea de valori distincte via **DISTINCT**

ordonarea rezultatelor prin **ORDER BY**

returnarea de sub-secvențe cu **LIMIT** și **OFFSET**

precizarea construcțiilor opționale prin **OPTIONAL**

sparql: limbaj de interogare

Datele interogate pot fi filtrate via clauza **FILTER**

expresiile de filtrare includ:

variabile

operatori aritmetici/logici

funcții (predefinite sau specificate de programator)

tipuri de date definite de XML Schema

sparql: limbaj de interogare

Funcții predefinite

de testare: **bound (), isBlank (), isLiteral (), isURI ()**

de conversie: **str (), lang (), datatype ()**

altele: **sameTerm (), langMatches (), regex ()**

sparql: limbaj de interogare

FILTER (?anul != 2)

FILTER (?dataComanda < "2025-12-31T22:00:00Z"^^xsd:date)

FILTER (?pop > 15000000 &&
langMatches (lang (?numeStat), "EN"))

FILTER (?foss = 'Yes' && ?categ = 'Database' &&
(?limbaj = 'PHP' || regex (?limbaj, "^C")))

```
SELECT ?numeStudenta
WHERE {
    ?persoana foaf:gender ?gen ;
              foaf:firstName ?numeStudenta .
  FILTER regex (?gen, "^female", "i") .
}
```

numeStudenta
"Ana"@ro
"Diana"@ro
"Cristina"@ro

```

SELECT ?numeStudenta
WHERE {
    ?persoana foaf:gender ?gen ;
              foaf:firstName ?numeStudenta .
  FILTER regex (?gen, "^female", "i") .
}

```

numeStudenta
"Ana"@ro
"Diana"@ro
"Cristina"@ro

Depth 1 RDF.Query.Algebra.**Select** Start

Depth 2 RDF.Query.Algebra.**Join** Start

Join on { **?persoana** }

Depth 3 RDF.Query.Algebra.**Filter** Start

Depth 4 RDF.Query.Algebra.**Bgp** Start

?persoana <<http://xmlns.com/foaf/0.1/gender>> **?gen** .

Depth 3 RDF.Query.Algebra.**Bgp** End

Depth 2 RDF.Query.Algebra.**Filter** End

Depth 1 RDF.Query.Algebra.**Join** End

Depth 0 RDF.Query.Algebra.**Select** End

explicarea interogării ce
oferă numele entităților
de gen feminin

```

SELECT DISTINCT ?studentSub22 ?titulatura
WHERE {
    ?persoana foaf:age ?ani ;
              foaf:firstName ?studentSub22
    OPTIONAL { ?persoana foaf:title ?titulatura . }
    FILTER (?ani <= 22)
}

```

studentSub22	titulatura
"Ana"@ro	"Bachelor of Science"@en
"Cristina"@ro	"Bachelor of Science"@en
"Diana"@ro	

numele și, eventual, titulatura persoanelor mai tinere

sparql: limbaj de interogare

Nodurile blanc într-o interogare
joacă rol de variabile anonime

```
SELECT DISTINCT ?nume ?ani ?prof
```

```
WHERE {
```

```
  [ ] foaf:knows ?persoana .
```

```
  ?persoana foaf:firstName ?nume ;
```

```
    rel:knowsByReputation
```

```
      <http://www.w3.org/People/Berners-Lee/card#i> ;
```

```
    foaf:age ?ani .
```

```
  ?prof s:examineaza ?persoana
```

```
}
```

s-a recurs la
vocabularul
relationship

nume	ani	prof
"Bogdan"@ro	23	s:busaco
"Cristina"@ro	21	s:busaco
"Bogdan"@ro	23	s:dlucanu

numele și vârsta cunoscuților care-l „știu” pe Sir Tim Berners-Lee după reputație și sunt examinați de profesori

sparql: limbaj de interogare

Crearea unui graf RDF se poate realiza prin **CONSTRUCT**

```
CONSTRUCT (?s ?p ?o)
WHERE {
    ?prop rdf:subject ?s .
    ?prop rdf:predicate ?p .
    ?prop rdf:object ?o .
    web:Wikipedia s:afirma ?prop .
}
```

determinarea tuturor declarațiilor reificate
exprimate de Wikipedia

sparql: limbaj de interogare

Operația **ASK** – exemplu:
aflarea răspunsului la întrebarea
„Fluviul Amazon este mai lung decât Nilul?”

PREFIX dbp: <<http://dbpedia.org/property/>>

ASK {

<http://dbpedia.org/resource/Amazon_River>

dbp:length ?lungAmazon .

<<http://dbpedia.org/resource/Nile>> dbp:length ?lungNil .

FILTER (?lungAmazon > ?lungNil) .

}

sparql: limbaj de interogare

Întrebare: „Fluviul Amazon este mai lung decât Nilul?”

Răspuns: DA – *true*



1

▼

PREFIX dbp: <http://dbpedia.org/property/>

2

▼

ASK {

3

<http://dbpedia.org/resource/Amazon_River> dbp:length ?lungAmazon .

4

<http://dbpedia.org/resource/Nile> dbp:length ?lungNil .

5

FILTER (?lungAmazon > ?lungNil) .

6

}





Table

Response

Pivot Table

Google Chart

Geo



 True

sparql: limbaj de interogare

Obținerea de șabloane de graf alternative
se realizează via clauza **UNION**

```
{ ?nod voc:hasName ?nume }  
      UNION  
      { ?nod vcard:FN ?nume }
```

```

SELECT ?cunoscutSauRudaCuCristinaExaminatDoarDeDL
WHERE {
  { ?pers foaf:knows s:cristina ;
    foaf:firstName ?cunoscutSauRudaCuCristinaExaminatDoarDeDL
  }
  UNION
  { ?pers rel:siblingOf s:cristina ;
    foaf:firstName ?cunoscutSauRudaCuCristinaExaminatDoarDeDL
  }
  s:dlucanu s:examineaza ?pers
}

```

cunoscutSauRudaCuCristinaExaminatDoarDeDL
"Bogdan"@ro
"Diana"@ro

resursele cunoscute sau rude cu o persoană
și examinate doar de un anumit profesor

sparql: limbaj de interogare

Întrebare:
am putea realiza interogări SPARQL
asupra mai multor grafuri RDF?

sparql: limbaj de interogare

O interogare poate fi aplicată:

grafului implicit (*default graph*)

unuia sau mai multor grafuri externe,
identificate via URI (*named graphs*)

fuziunii dintre graful implicit și cele externe

sparql: limbaj de interogare

Graful RDF de intrare poate fi specificat via URI

exemplu:

FROM <<http://planetrdf.com/bloggers.rdf>>

sparql: protocol

Pune la dispoziție o modalitate de realizare a interogărilor SPARQL prin invocarea de servicii Web – *SPARQL end-points*

recomandări ale Consorțiului Web (2008, 2013)

www.w3.org/TR/rdf-sparql-protocol/

www.w3.org/TR/sparql11-protocol/

în lucru (aug. 2025): <https://w3c.github.io/sparql-protocol/spec/>

sparql: protocol

Serviciile SPARQL pot fi descrise și invocate
via tehnologiile serviciilor Web clasice – *i.e.*, prin REST

un punct terminal SPARQL – generic sau particular –
acceptă cereri și trimite rezultate via HTTP(S)

rezultatul poate fi oferit într-o suită de formate:
CSV, HTML, JSON(-LD), RDF (N3, Turtle), XML etc.

sparql: protocol

Serviciile SPARQL pot fi descrise și invocate
via tehnologiile serviciilor Web clasice – *i.e.*, prin REST

un punct terminal SPARQL – generic sau particular –
acceptă cereri și trimite rezultate via HTTP(S)

exemple concrete:

DBpedia – dbpedia.org/sparql/

Wikidata – query.wikidata.org

European Data – data.europa.eu/data/sparql

The Nobel Prize data – data.nobelprize.org/sparql

sparql: protocol

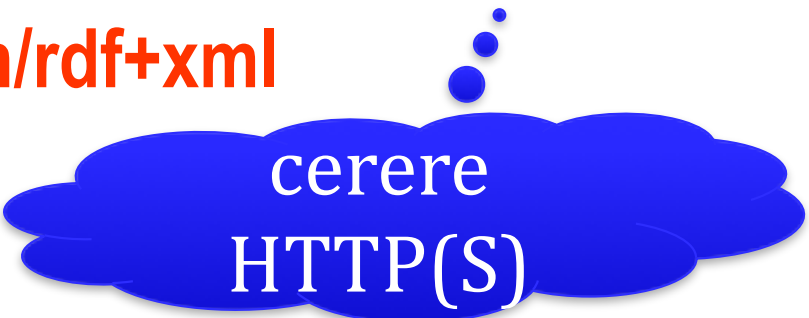
Invocarea unui *end-point* SPARQL (serviciu REST) recepționând ca date de intrare o interogare:

GET /sparql/?query=*interogare*&graph-uri=
<https://www.truefacts.info/dataset?type=rdf> HTTP/1.1

Host: www.undeva.org

User-agent: sparql-client/1.0

Accept: **application/turtle**, **application/rdf+xml**



cerere
HTTP(S)

sparql: protocol

Un posibil răspuns din partea serviciului:

HTTP/1.1 200 OK

Date: Tue, 11 Nov 2025 12:33:00 GMT

Server: Apache

Connection: close

Content-Type: application/turtle

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

<http://banchiza.web/Tux> rdf:type <http://dbpedia.org/resource/Penguin> .

...



mesaj de
răspuns HTTP

sparql: protocol

Maniera de serializare a rezultatelor interogărilor SPARQL este standardizată

SPARQL Query Results JSON Format (2013)

www.w3.org/TR/sparql11-results-json/

SPARQL Query Results XML Format (2013)

www.w3.org/TR/rdf-sparql-XMLres/

în lucru (aug 2025): www.w3.org/TR/sparql12-results-json/

sparql: instrumente

Procesoare SPARQL – exemplificări:

AllegroGraph (Java)

ARC, **EasyRDF** (PHP)

ARQ (Java – oferit de Apache Jena)

dotNetRDF (C#)

Rasqual (C, cu diverse portări – bazat pe Redland)

RDF::Query (Perl)

sparql (Ruby)

sparql-p (Python)

SPARQL.js, **rdf-store.js**, **MarkLogic Client API** (Node.js)

de explorat și www.w3.org/wiki/SparqlImplementations

sparql: instrumente

Clienți – exemple:

sparql-client (Python) – acces la un *end-point* SPARQL
pypi.org/project/sparql-client/

sparql-http-client (Node.js)
rdf-ext.github.io/sparql-http-client/

SparqlGUI (.NET) – instrumente SPARQL pentru *desktop*
github.com/dotnetrdf/dotnetrdf/wiki/UserGuide-Tools

SPARQL Playground, YASGUI, SPARQLer
aplicații Web de editare/testare direct în *browser*

Twinkle: SPARQL Tools

File Edit Query

Select Query Task

- General
 - Write Simple Query
- In Memory
 - Planet RDF Feed & Blogroll Periodic Table
- Inferencing
 - Planet Feed (RDFS)
- Persistent Stores
- Remote Services
 - Reyvu.com
 - GovTrack.us
 - DBpedia.org

*Untitled

Save Run Cancel

Base URI

Data URL file:/Users/busaco/Desktop/sweet.rdf File...

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX g: <http://enradsheats.google.com/feeds/calls/018232457320750206107_1700652366446048707/ed6...

```
SELECT ?nume ?url ?limbaj WHERE {  
  [ g:label ?nume ; g:URL ?url ; g:FOSS ?foss ; g:Category ?categ ;  
    g:Language ?limbaj ] .  
  FILTER ( ?foss = 'Yes' && ?categ = 'Database/Datastore' &&  
    ( ?limbaj = 'PHP' || regex ( ?limbaj, "^C" ) ) ) .  
} ORDER BY ?limbaj
```

nume	url	limbaj
3store	http://www.aktors.org/technologies/3store/	C / C++
Brahms	http://lsdis.cs.uga.edu/projects/semdis/brahms/	C / C++
OpenLink Virtuoso	http://virtuoso.openlinksw.com/wiki/main	C / C++
RDFStore	http://rdfstore.sourceforge.net/	C / C++
Soprano	http://soprano.sourceforge.net/	C / C++
SemWeb	http://razor.occams.info/code/semweb/	C#
RAP NetAPI	http://www.wiwiss.fu-berlin.de/suhl/bizer/rdfapi/tutorial/netapi.html	PHP

text table

Selected data file /Users/busaco/Desktop/sweet.rdf

Twinkle – interogări asupra unui document RDF local



http://dbpedia.org/sparql

1

2

3

4

5

6

```
SELECT DISTINCT ?abilitati
WHERE {
  [ a <http://dbpedia.org/ontology/FictionalCharacter> ;
    <http://dbpedia.org/property/abilities> ?abilitati ] .
}
LIMIT 30
```





Table

Response

Pivot Table

Showing 1 to 30 of 30 entries (in 0.324 seconds)

abilitati

1

2

3

4

5

6

"Supernatural strength, speed, stamina, agility, reflexes, and rapid healing."^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>

"Enhanced intuition, prophetic dreams."^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>

<http://dbpedia.org/resource/Precognition>

[http://dbpedia.org/resource/Magic_\(paranormal\)](http://dbpedia.org/resource/Magic_(paranormal))

[http://dbpedia.org/resource/Magician_\(fantasy\)](http://dbpedia.org/resource/Magician_(fantasy))

"Genius-level intellect"^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>

SELECT DISTINCT ?abilitati

WHERE {

[a <http://dbpedia.org/ontology/FictionalCharacter> ;
<http://dbpedia.org/property/abilities> ?abilitati] . }

LIMIT 30

listarea abilităților
personajelor fictive

YASGUI (*Yet Another SPARQL GUI*): github.com/TriplyDB/Yasgui
instrument Web avansat de interogare SPARQL

Your SPARQL query

Add common prefixes

```
1 PREFIX keywords: <http://purl.uniprot.org/keywords/>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX up: <http://purl.uniprot.org/core/>
4 SELECT DISTINCT ?link
5 WHERE
6 {
7   ?protein a up:Protein .
8   ?protein up:classifiedWith keywords:5 .
9   ?protein rdfs:seeAlso ?link .
10  ?link up:database ?db .
11  ?db up:category '3D structure databases'
12 }
```

About

This SPARQL endpoint contains all UniProt data. It is free to access and supports the [SPARQL 1.1 Standard](#).

There are 217,505,202,099 triples in this release (2025_04). The query timeout is 45 minutes. All triples are available in the default graph. There are 22 named graphs.

Documentation

1. [Classes and predicates defined by the UniProt consortium](#)
2. [Tutorial on using SPARQL with UniProt](#)
3. [Statistics and diagrams](#)
4. [Example queries](#)

Examples

1. Select all taxa from the [UniProt taxonomy](#) [Use](#)
2. Select all bacterial taxa and their scientific name from the [UniProt taxonomy](#) [Use](#)
3. Select all UniProtKB entries, and their organism and amino acid sequences (including isoforms), for *E. coli K12* and all its strains [Use](#)
4. Select the UniProtKB entry with the mnemonic 'A4_HUMAN' [Use](#)
5. Select a mapping of UniProtKB to PDB entries using the UniProtKB cross-references to the [PDB database](#) [Use](#)
6. Select all cross-references to external databases of the category '3D structure databases' of UniProtKB entries that are classified with the keyword 'Acetoin biosynthesis (KW-0005)' [Use](#)

interogări SPARQL specifice:
proiectul **UniProt**
(*Universal Protein Resource*)
sparql.uniprot.org

sparql: instrumente

Puncte terminale (*end-points*) SPARQL

www.w3.org/wiki/SparqlEndpoints

+

seturi de date RDF deschise – exemplificări:

resurse enciclopedice – **DBpedia** (*snapshot releases*)

www.dbpedia.org/resources/snapshot-release/

istorie – **World War I as Linked Open Data**

www.ildf.fi/dataset/ww1lod/

numismatică – inițiativa **Nomisma**

nomisma.org/datasets + nomisma.org/sparql/

muzică clasică – **DBtune**

dbtune.org/classical/



Câteva exemplificări concrete?

Interogarea SPARQL asupra DBpedia: informații despre locul de naștere a lui A. I. Cuza

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?locNastere ?dataNastere WHERE {
  ?persoana rdfs:label      "Alexandru Ioan Cuza"@en ;
            dbo:birthPlace  ?locNastere ;
            dbo:birthDate   ?dataNastere .
}
```

de observat
diferitele
granularități

	locNastere	dataNastere
	http://dbpedia.org/resource/Bârlad	"1820-03-20"^^xsd:date
	http://dbpedia.org/resource/Huși	"1820-03-20"^^xsd:date
3	http://dbpedia.org/resource/Moldavia	"1820-03-20"^^xsd:date

```

1 PREFIX foaf: <http://xmlns.com/foaf/0.1/>
2 PREFIX dbp: <http://dbpedia.org/property/>
3
4 SELECT ?nume ?ocupatie
5 WHERE {
6     ?persoana foaf:name ?nume ;
7             dbp:occupation ?ocupatie .
8     FILTER regex (?nume, "terson$")
9 }
10 ORDER BY ?nume
11 LIMIT 7

```

interogarea SPARQL
asupra DBpedia:
ocupația persoanelor
cu un anumit sufix
al numelui



Table

Response

Pivot Table

Google Chart

Geo



Showing 1 to 7 of 7 entries (in 0.695 seconds)

	nume	ocupatie
1	"Abi Peterson"@en	http://dbpedia.org/resource/Detective
2	"Abi Peterson"@en	http://dbpedia.org/resource/Businessperson
3	"Abi Peterson"@en	"Student"^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>
4	"Abi Peterson"@en	"Factory worker"^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>

clase de
resurse

SPARQL Query Editor About Tables ▾ Conductor Facet Browser Permalink

Default Data: http://dbpedia.org

Query Text: select distinct

ser: SPARQL

```
# obținerea de informații despre România
SELECT ?info
WHERE {
  <http://dbpedia.org/resource/Romania> rdfs:comment ?info
}
```

Auto HTML

- ✓ HTML (Faceted Browsing Links)
- Spreadsheet
- XML
- JSON
- Javascript
- Turtle**
- RDF/XML
- N-Triples
- CSV
- TSV

- ☒ Strict checking of void variables
- ☒ Strict checking of variable names used in multiple clauses but not logically connected to each other
- ☐ Suppress errors on wrong geometries and errors on geometrical operators (failed operations will return NULL)
- ☐ Log debug info at the end of output (has no effect on some queries and output formats)
- ☐ Generate SPARQL compilation report (instead of executing the query)

acces la DBpedia via interfața Web a serviciului SPARQL
ce recurge la sistemul Virtuoso

```

SELECT DISTINCT ?Concept
FROM <http://dbpedia.org>
WHERE {
  _:OrgnBNanon_6_1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> ?Concept .
  FILTER ( ?TmpBNdefault_6_2 IS MEMBER OF ( <SPECIAL::sql:RDF_GRAPH_GROUP_LIST_GET>( <http://dbpedia.org>, NULL, 0, N
LIMIT 100

```

SPARQL query translated to SQL

For security reasons, code responsible for graph-level security is not generated and some account-specific data is deliberately obfuscated.

```

SELECT  __ro2sq ( "s_5_0_rbc"."Concept") AS "Concept"
FROM (SELECT DISTINCT TOP 100 "s_5_0_t0"."0" AS "Concept"
FROM DB.DBA.RDF_QUAD AS "s_5_0_t0"
WHERE
  "s_5_0_t0"."P" = __i2idn ( __bft( 'http://www.w3.org/1999/02/22-rdf-syntax-ns#type' , 1))
AND
  ( "s_5_0_t0"."G" in /* =item-in-vector */ ( DB.DBA.RDF_GRAPH_GROUP_LIST_GET (
    __bft ( __bft( 'http://dbpedia.org' , 1), 1),
    NULL,
    0,
    NULL,
    NULL,
    1)))
OPTION (QUIETCAST)) AS "s_5_0_rbc"

```

intern, interogarea SPARQL
este optimizată & convertită
în SQL spre a fi executată

SQL execution plan

```

{
  Precode:
    0: DB.DBA.RDF_GRAPH_GROUP_LIST_GET$29 := Call DB.DBA.RDF_GRAPH_GROUP_LIST_GET (<tag 182 flag 1 c http://dbpedia.org>,
    7: chash_in_init$31 := Call chash_in_init ( 244 , chash_in_tree$33, 0 , 0 , DB.DBA.RDF_GRAPH_GROUP_LIST_GET$29)
    12: BReturn 0
  Subquery i#34
  {
    From RDF_QUAD 1.9e+07 rows
    Key RDF QUAD ASC (s 5 0 t0.0$39, s 5 0 t0.G$38)

```

Obținerea datelor privind personalitățile
(persoane cu o anumită ocupație, eventual monarhi)
care au numele începând cu șirul „Ad”

```
SELECT DISTINCT ?personalitate ?nume ?dataMoarte
WHERE {
    ?personalitate a
        <http://umbel.org/umbel/rc/PersonWithOccupation> .
    OPTIONAL { ?personalitate a dbo:Monarch . }
    ?personalitate dbp:deathDate ?dataMoarte .
    ?personalitate rdfs:label ?nume .
    FILTER ( lang (?nume)="en" && regex (?nume, "^Ad")) .
}
ORDER BY desc (?nume) ?dataMoarte
LIMIT 10
```



http://dbpedia.org/sparql



```
2 PREFIX dbo: <http://dbpedia.org/ontology/>
3 PREFIX dbp: <http://dbpedia.org/property/>
4 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
5 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
6
7 SELECT DISTINCT ?personalitate ?nume ?dataMoarte
8 WHERE {
9     ?personalitate a
10         <http://umbel.org/umbel/rc/PersonWithOccupation> .
11     OPTIONAL { ?personalitate a dbo:Monarch . }
12     ?personalitate dbp:deathDate ?dataMoarte .
13     ?personalitate rdfs:label ?nume .
14     FILTER (lang (?nume) = "en" && regex (?nume, "^Ad")) .
15 }
16 ORDER BY desc (?nume) ?dataMoarte
17 LIMIT 10
```



Table

Response

Pivot Table

Google Chart

Geo



Showing 1 to 4 of 4 entries (in 1.549 seconds)

Search:

Show

All



entries

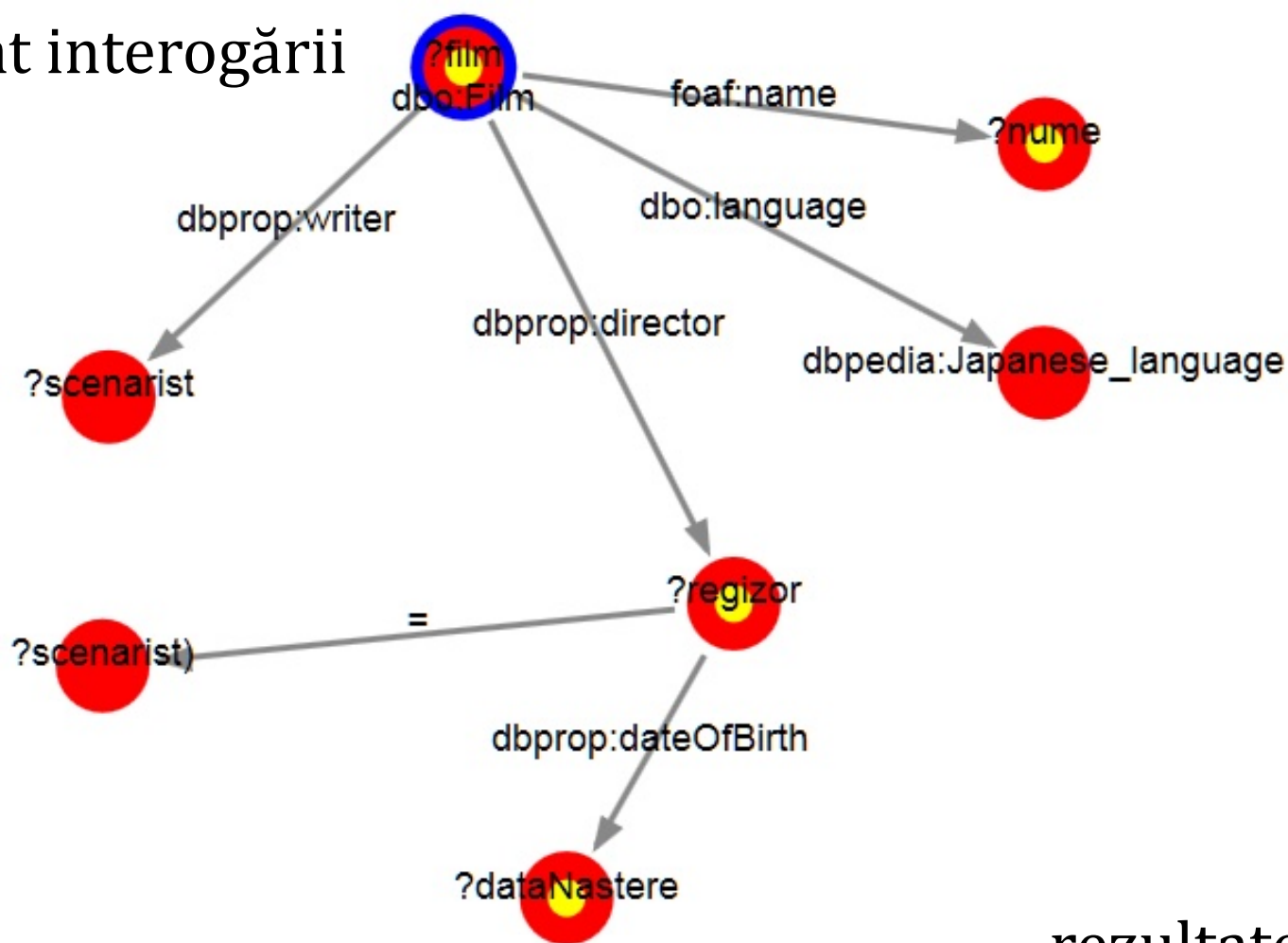
	personalitate	nume	dataMoarte
1	http://dbpedia.org/resource/Adolphe_Niel	"Adolphe Niel"@en	"1869-08-13"^^xsd:date
2	http://dbpedia.org/resource/Adolph_Malan	"Adolph Malan"@en	"1963-09-17"^^xsd:date
3	http://dbpedia.org/resource/Adelaide_of_Maurienne	"Adelaide of Maurienne"@en	"--11-18"^^xsd:gMonthDay
4	http://dbpedia.org/resource/Adela_of_Normandy	"Adela of Normandy"@en	"--03-08"^^xsd:gMonthDay

Lista filmelor vorbite în limba japoneză, regizate și
– totodată – scrise de persoane născute după anul 1960

PREFIX dbo: <<http://dbpedia.org/ontology/>>
PREFIX dbp: <<http://dbpedia.org/property/>>
PREFIX foaf: <<http://xmlns.com/foaf/0.1/>>

```
SELECT ?film ?nume ?regizor ?dataNastere WHERE {  
  ?film a dbo:Film ;  
    dbo:language <http://dbpedia.org/resource/Japanese\_language> ;  
    dbp:director ?regizor ;  
    dbp:writer ?scenarist ;  
    foaf:name ?nume .  
  ?regizor dbp:dateOfBirth ?dataNastere .  
  FILTER ((?dataNastere >= 1960) && ?regizor = ?scenarist) .  
}  
ORDER BY ?nume  
LIMIT 7
```

graful asociat interogării



rezultatele obținute

film	nume	regizor	dataNastere
dbpedia:The Taste of Tea	(茶の味 Cha no Aji)	dbpedia:Katsuhito Ishii	1966
dbpedia:A Stranger of Mine	A Stranger of Mine運命じゃない人	dbpedia:Kenji Uchida (film director)	1972
dbpedia:Kokkuri-san (film)	Kokkuri-san	dbpedia:Takahisa Zeze	1960
dbpedia:Linda Linda Linda	Linda Linda Linda	dbpedia:Nobuhiro Yamashita	1976
dbpedia:Moon Child (2003 film)	Moon Child	dbpedia:Takahisa Zeze	1960
dbpedia:Purple Butterfly	Purple Butterfly	dbpedia:Lou Ye	1965
dbpedia:Slayers Premium	Slayers Premium	dbpedia:Junichi Sato	1960

sparql: detectarea inconsistențelor

Fie interogarea SPARQL asupra DBpedia privind data de naștere a lui Mihai Eminescu

PREFIX rdfs: <<http://www.w3.org/2000/01/rdf-schema#>>

PREFIX dbo: <<http://dbpedia.org/ontology/>>

```
SELECT ?dataNastere WHERE {  
  ?persoana rdfs:label      "Mihai Eminescu"@en ;  
            dbo:birthDate   ?dataNastere .  
}
```

dataNastere
"1849-12-20"^^xsd:date
"1850-01-15"^^xsd:date

???

SPARQL oferă premisele verificării corectitudinii datelor
context mai larg: *knowledge quality*

Detectarea resurselor DBpedia care prezintă adrese Web specificate eronat

PREFIX foaf: <<http://xmlns.com/foaf/0.1/>>

PREFIX dbp: <<http://dbpedia.org/property/>>

SELECT ?subject ?url

WHERE {

{ ?subject foaf:page ?url . } UNION { ?subject dbp:siteweb ?url . }

FILTER (isLITERAL (?url))

}

subject	url
http://dbpedia.org/resource/Agglomeration_community_of_the_Saumur_Loire_development	"http://www.agglo-saumur.fr/"^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>
http://dbpedia.org/resource/Agglomeration_community_of_Rouen	"Official site of the Agglo-Rouen.fr"^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>
http://dbpedia.org/resource/Communauté_de_communes_du_Val_de_Noye	apare rdfs:Literal în loc de rdfs:Resource
http://dbpedia.org/resource/Communauté_de_communes_du_canton_de_Conty	
	"www.cccconty.com"^^<http://www.w3.org/1999/02/22-rdf-syntax-ns#langString>

Titlul lucrărilor în română ce au fost traduse în italiană

```
SELECT DISTINCT ?titlu ?data ?editor WHERE {  
  [] foaf:focus ?lucrare .  
  ?lucrare rdarerelationships:expressionOfWork ?exp .  
  ?lucrare dcterms:language <http://id.loc.gov/vocabulary/iso639-2/rum> .  
  ?manif rdarerelationships:expressionManifested ?exp .  
  OPTIONAL { ?exp bnfroles:r680 ?trad ;  
    dcterms:language <http://id.loc.gov/vocabulary/iso639-2/ita> } .  
  OPTIONAL { ?manif dcterms:date ?data }  
  OPTIONAL { ?manif dcterms:title ?titlu }  
  OPTIONAL { ?manif dcterms:publisher ?editor }  
}  
LIMIT 7
```

} date opționale

Bibliothèque Nationale de France – acces la date publice
data.bnf.fr/semanticweb

titlu	data	editor
Marxism oligarhic, contribuție la problema dezvoltării capitaliste a României	1926	București : I. Brănișteanu , 1926
Le lit de Procuste : pour voix, clarinette alto et piano	1971	Paris : Salabert , cop. 1971
4 poèmes de Nichita Stănescu : pour voix et piano	[s.d.]	Paris : Salabert , [s.d.]
Paul Le Flem : Leben und Werk der ersten Kompositionsperiode	1985	Tübingen : [G. Bernard-Krauss] , 1985
Evreii din România : 1919-1938, de la emancipare la marginalizare	2000	București : Hasefer , 2000
Hugoliade	1982	[Paris] : Gallimard , 1982
Teatru	1970	București : Minerva , 1970

rezultatele oferite de serviciul SPARQL
data.bnf.fr/sparql/

⚠ a se remarca problemele
vizând diacriticele și datele temporale

```

SELECT ?label ?lat ?long WHERE {
  ?loc spatial:nearby (47.1585 27.6014 30 'km') ;
    geo:lat ?lat ; geo:long ?long .
  ?mint a nmo:Mint ; geo:location ?loc ;
    skos:prefLabel ?label .
FILTER langMatches (lang(?label), 'en')
}

```

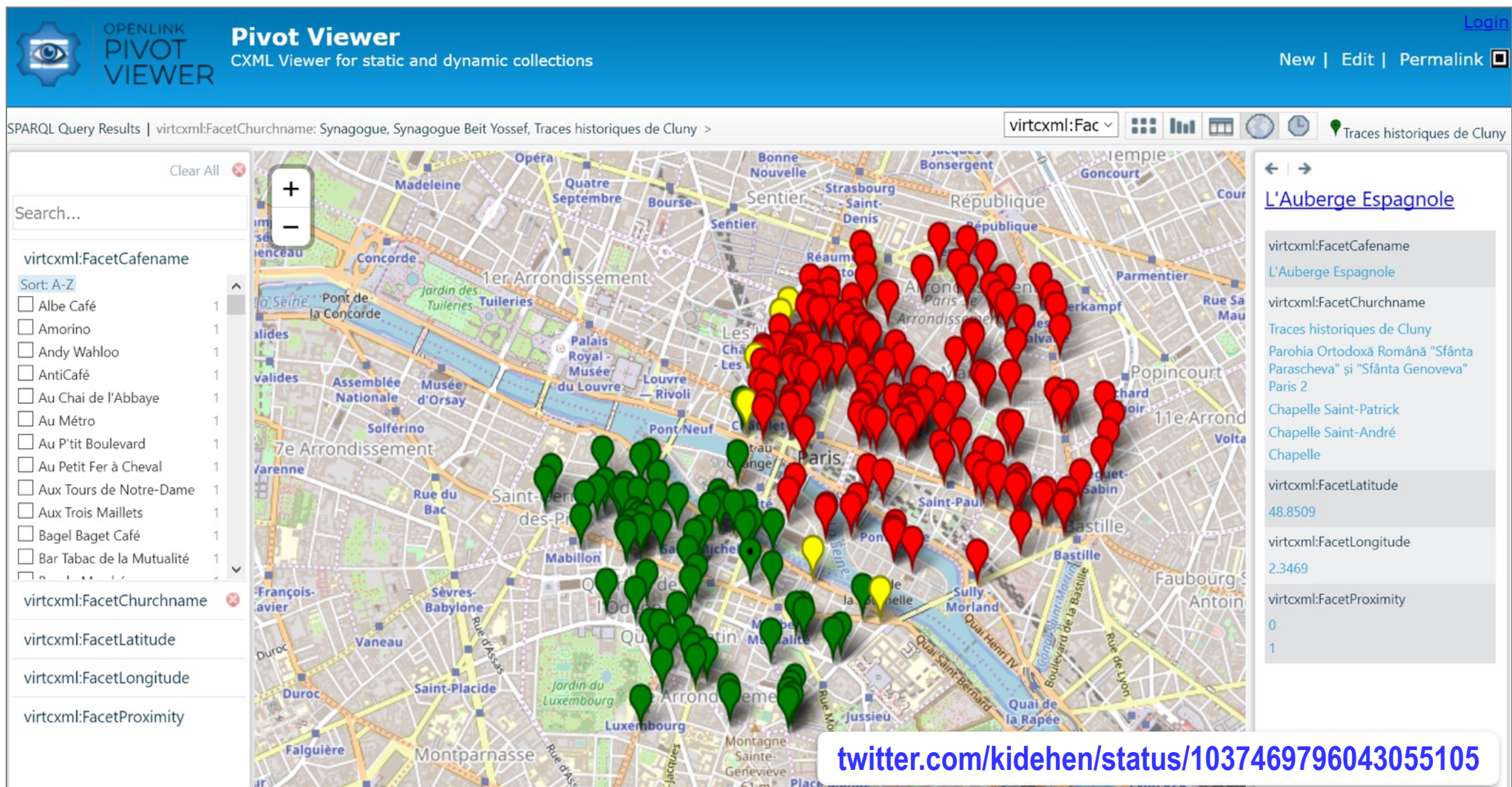
acces la date numismatice:
nomisma.org/sparql
 folosind extensii geo-spațiale
github.com/galbiston/geosparql-fuseki

```

{ "head": { "vars": [ "label", "lat", "long" ] },
  "results": { "bindings": [ {
    "label": { "type": "literal", "xml:lang": "en", "value": "Rome" },
    "lat": { "type": "literal",
      "datatype": "http://www.w3.org/2001/XMLSchema#decimal",
      "value": "41.9" },
    "long": { "type": "literal",
      "datatype": "http://www.w3.org/2001/XMLSchema#decimal",
      "value": "12.5" }
    ... ] } }

```

... răspuns JSON oferit

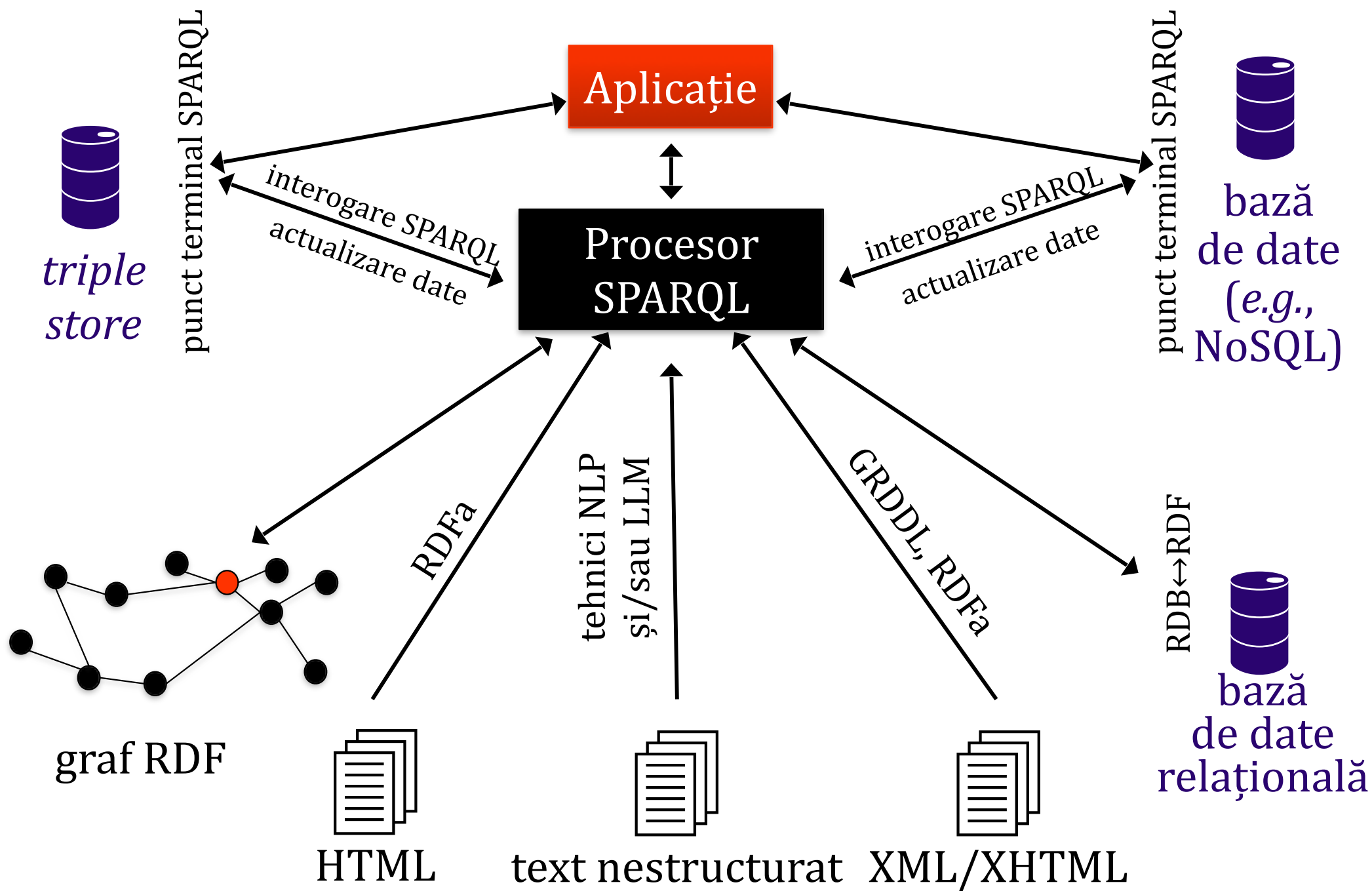


vizualizarea structurată a datelor oferite de o interogare
– aici, cafenele în proximitatea lăcașelor de cult –
via **OpenLink HTML5 Pivot Viewer**
github.com/openlink/html5pivotviewer

sparql: recapitulare

O interogare SPARQL constă în:

- declarații de prefixe – vocabulare desemnate de URL
- specificări ale seturilor de date (grafuri RDF) interogate
- clauză (*result clause*) identificând datele rezultate
- un șablon (*query pattern*) reprezentând termeni liberi
- modificatori (*query modifiers*) ce oferă posibilități de fragmentare, sortare, rearanjare a rezultatelor interogării



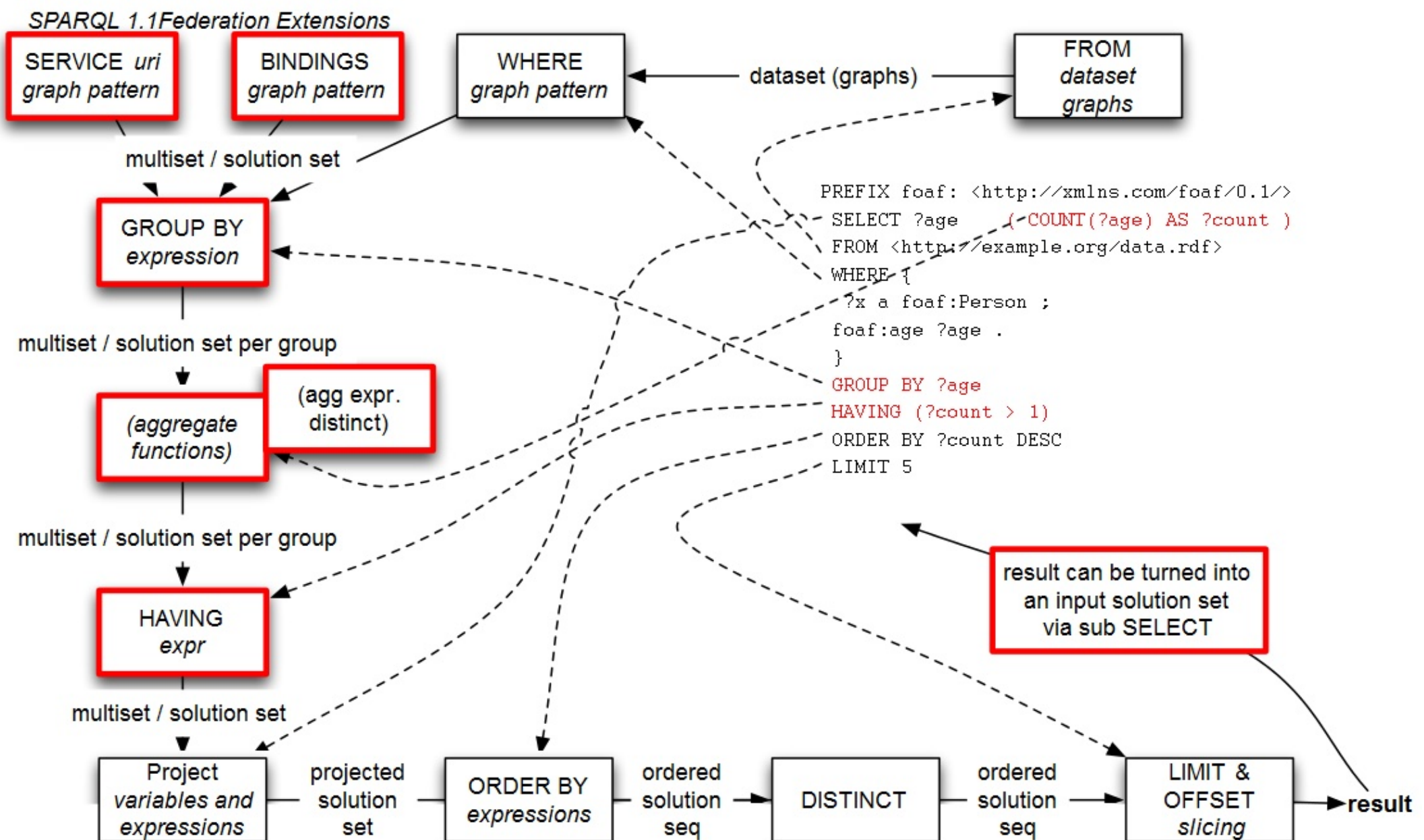
SPARQL – privire de ansamblu (după Ivan Herman)

sparql 1.1

Recomandare W3C (2013)

oferă mijloace pentru exprimarea:
actualizărilor de date (*updates*)
negațiilor (*negation*)
agregărilor (*aggregates*)
proiecțiilor (*projected expressions*)
sub-interogărilor (*subqueries*)

www.w3.org/TR/sparql11-query/



anatomia unei interogări SPARQL 1.1
conform David Backett (2011)

funcții de agregare: **AVG, COUNT, MIN, MAX, SUM,...**

```
SELECT ?prof (COUNT (DISTINCT ?stud) AS ?studenti)
WHERE {
    ?prof a ac:Student_Adviser ;
    s:examineaza ?stud
} GROUP BY ?prof
```

prof	studenti
s:busaco	4
s:dlucanu	2

obținerea datelor privind profesorii îndrumători și
numărul studenților examinați de fiecare

sparql 1.1

Posibilitatea utilizării funcțiilor de proiecție:

```
SELECT (CONCAT(?prenume, ' ', ?nume) AS ?numeComplet)
WHERE {
    ?persoana schema:givenName ?prenume ;
              schema:familyName ?nume ;
              :interestedBy      :Web , :HCI , :Art ,
                                <https://www.wikidata.org/wiki/Q638> .
}
```



obținerea numelui complet al persoanelor
interesate de diverse domenii

Funcții utile:

operații logice: **IF()** – *e.g.*, **IF(?nota > 9, "super", "promovat")**

procesări numerice: **ABS() ROUND() CEIL() FLOOR() RAND()
COUNT() MIN() MAX() AVG() SUM()**

prelucrarea șirurilor de caractere: **STRLEN() SUBSTR()
LCASE() UCASE() STRSTARTS() STREND() CONTAINS()
STRBEFORE() STRAFTER() REPLACE()**

privitoare la dată și timp: **NOW() YEAR() MONTH() DAY()
HOURS() MINUTES() SECONDS() TIMEZONE() TZ()**

generări de *hash*: **MD5() SHA1() SHA256()** etc.

crearea unui nod blank: **BNODE ()**

MINUS exprimă negația (diferența mulțimilor de triple)

MINUS

“removing matches based on evaluation of graph patterns”

versus

NOT EXISTS

“testing whether a pattern exists in the data”

```
SELECT * {  
  ?s ?p ?o .  
  MINUS { [ ] nobel:category  
    <http://data.nobelprize.org/resource/category/Peace> }  
  MINUS { [ ] dbo:birthPlace  
    <http://data.nobelprize.org/resource/country/Canada> }  
}
```

```
SELECT ?nume
WHERE {
  ?persoana foaf:firstName ?nume
  FILTER (
    NOT EXISTS { ?persoana rel:knowsByReputation [] }
  )
}
```

nume
"Ana"@ro
"Diana"@ro
"Dorel"@ro

utilizarea construcției **NOT EXISTS**
aici: numele persoanelor care nu cunosc după reputație
pe cineva important

sparql 1.1

Specificarea căilor vizând proprietățile
(*property paths*)

potrivirile de triple iau în considerație
drumurile prin graful RDF
specificate sintactic via expresii similare XPath

a/b a $a|b$ a^* a^+ $a?$ $a\{min,max\}$ $a\{n\}$ $a\{min, \}$ $a\{,max\}$

unde **a** și **b** sunt URI-uri ce semnifică predicate

```
SELECT ?persoana
WHERE {
  s:dlucanu
  s:examineaza/(rel:closeFriendOf|rel:siblingOf)
  ?persoana
}
```

persoana
s:diana
s:cristina

determinarea persoanelor care sunt prieteni apropiați
sau rude cu un student examinat de un anumit profesor

Laureații Nobel născuți în România, ordonați după vârstă

```
SELECT DISTINCT ?nume ?varsta ?dataNastere ?dataMoarte
WHERE {
  ?laureat rdfs:label ?nume ;
    dbp:dateOfBirth ?dataNastere ;
    dbo:birthPlace
      <http://data.nobelprize.org/resource/country/Romania> .
  OPTIONAL { ?laureat dbp:dateOfDeath ?dataMoarte } .
  # vârstă curentă
  BIND(IF(BOUND(?dataMoarte), ?dataMoarte, NOW()) AS ?final)
  BIND(YEAR(?final) - YEAR(?dataNastere) -
    IF(MONTH(?final) < MONTH(?dataNastere) ||
      (MONTH(?final) = MONTH(?dataNastere) &&
        DAY(?final) < DAY(?dataNastere)), 1, 0 ) AS ?varsta )
}
ORDER BY DESC(?varsta)
LIMIT 5
```

BIND() asociază o valoare
unei variabile

```

7 SELECT DISTINCT ?nume ?varsta ?dataNastere ?dataMoarte
8 WHERE {
9     ?laureat rdfs:label ?nume ;
10             dbp:dateOfBirth ?dataNastere ;
11             dbo:birthPlace
12             <http://data.nobelprize.org/resource/country/Romania> .
13     OPTIONAL { ?laureat dbp:dateOfDeath ?dataMoarte } .
14     # vârsta curentă
15     BIND(IF(BOUND(?dataMoarte), ?dataMoarte, NOW()) AS ?final)
16     BIND(YEAR(?final) - YEAR(?dataNastere) -
17           IF(MONTH(?final) < MONTH(?dataNastere) ||
18             (MONTH(?final) = MONTH(?dataNastere) &&
19              DAY(?final) < DAY(?dataNastere)), 1, 0 ) AS ?varsta )
20 }
21 ORDER BY DESC(?varsta)
22 LIMIT 5

```



Table Response 4 results in 0.261 seconds

Simple view ☐ Ellipse ☒

Filter query results

Page size: 50

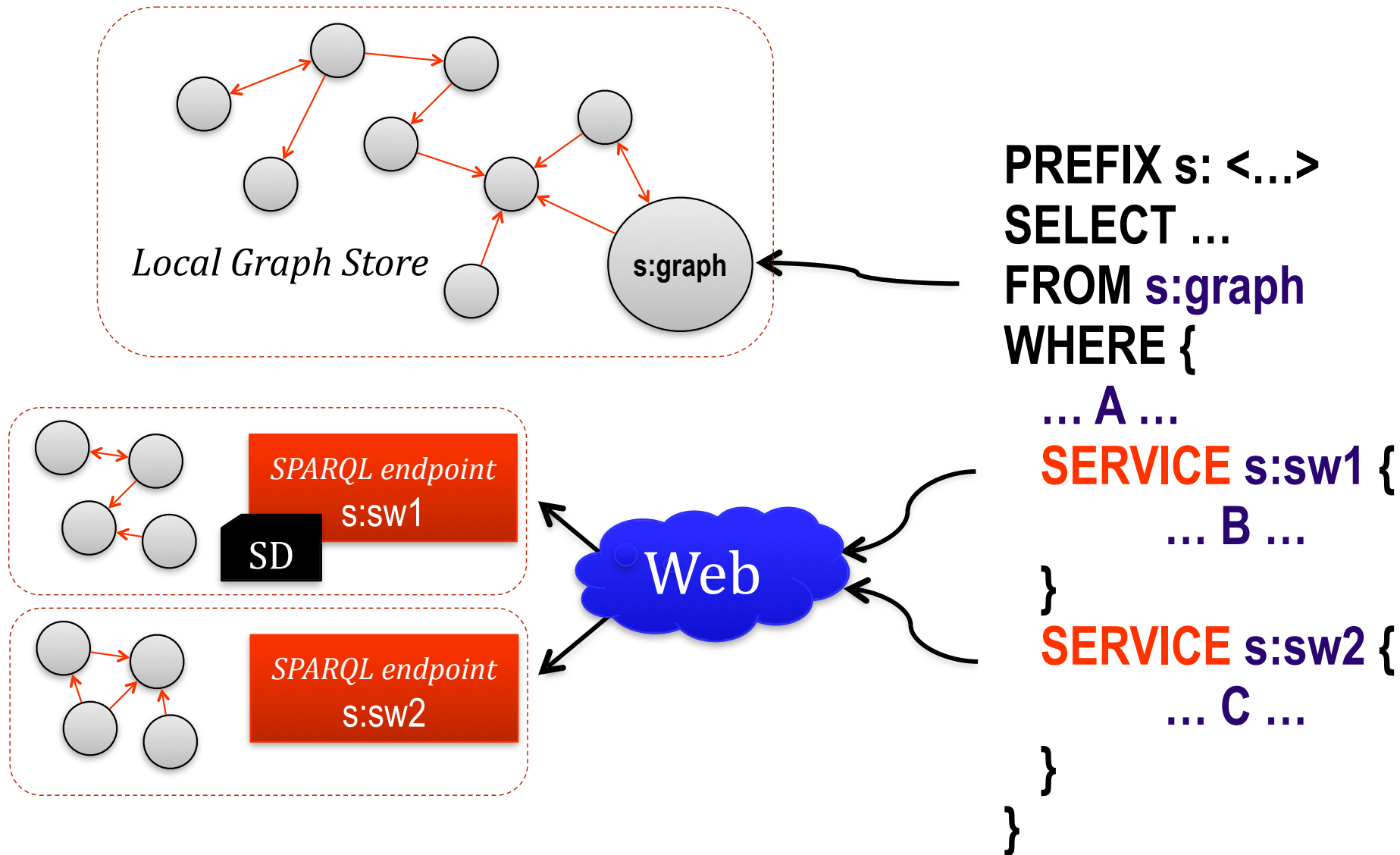


	nume	varsta	dataNastere	dataMoarte
1	George E. Pa...	"95"^^<http://www.w3.org/2001/XMLSchema...	"1912-11-19"^^<http://www.w3.org/2001/XMLSc...	"2008-10-07"^^<http://www.w3.org/2001/XMLS...
2	Elie Wiesel	"87"^^<http://www.w3.org/2001/XMLSchema...	"1928-09-30"^^<http://www.w3.org/2001/XML...	"2016-07-02"^^<http://www.w3.org/2001/XMLSc...
3	Herta Müller	"72"^^<http://www.w3.org/2001/XMLSchema...	"1953-08-17"^^<http://www.w3.org/2001/XMLS...	
4	Stefan W. Hell	"62"^^<http://www.w3.org/2001/XMLSchema...	"1962-12-23"^^<http://www.w3.org/2001/XMLS...	

tipuri de date
specificate explicit

detalii la data.nobelprize.org

Recurgerea la puncte terminale SPARQL multiple (*federated query*) – adaptare după Lee Feigenbaum



SD = *Service Description* – descrierea unui serviciu (folosind RDF)

Wikidata Query

Examples

Prefixes

Tools

Help

```

1 #added before 2016-10
2 SELECT DISTINCT ?city ?cityLabel ?mayor ?mayorLabel
3 WHERE
4 {
5     ?city wdt:P31/wdt:P279* wd:Q515 .    # find instances of subclasses of city
6     ?city p:P6 ?statement .              # with a P6 (head of government) statement
7     ?statement ps:P6 ?mayor .            # ... that has the value ?mayor
8     ?mayor wdt:P21 wd:Q6581072 .         # ... where the ?mayor has P21 (sex or gender) female
9     FILTER NOT EXISTS { ?statement pq:P582 ?x } # ... but the statement is not deprecated
10
11     # Now select the population value of the ?city
12     # (wdt: properties use only statements of "preferred" rank if any, usually meaning "current population")
13     ?city wdt:P1082 ?population .
14     # Optionally, find English labels for city and mayor:
15     SERVICE wikibase:label {
16         bd:serviceParam wikibase:language "en" .
17     }
18 }
19 ORDER BY DESC(?population)
20 LIMIT 10

```

Find

?city instance of any subclass of city with ?mayor sex or gender female

Show

?city head of government ?statement , ?statement head of government ?mayor , ?city population ?population

Limit 10

(P279) all instances of these items are instances of those items; this item is a class of that item. Not to be confused with Property:P31 (instance of).

?city wdt:P31/wdt:P279* wd:Q515

P31 = *instance of* (echivalentă cu `rdf:type`)

P279 = *subclass of* (`≡ rdfs:subClassOf`)

Q515 = *city* (`≡ http://dbpedia.org/ontology/City`)

Press [CTRL-SPACE] to activate auto completion.

Run

Clear

10 Results in 1178 ms

Display

Download

city	cityLabel	mayor	mayorLabel
wd:Q1490	Tokyo	wd:Q261703	Yuriko Koike
wd:Q1156	Mumbai	wd:Q18218029	Snehal Ambekar

exemplu concret: o interogare asupra **Wikidata**
www.wikidata.org/wiki/Wikidata:SPARQL_query_service

sparql 1.1

Un serviciu SPARQL poate avea atașată o descriere RDF
SPARQL 1.1 Service Description (W3C, 2013)

www.w3.org/TR/sparql11-service-description/

@prefix sd: <<http://www.w3.org/ns/sparql-service-description#>> .

<#SPARQL-endpoint>

a sd:Service ;

sd:endpoint <<http://dbpedia.org/sparql/>> ;

sd:supportedLanguage sd:SPARQL11Query ;

sd:resultFormat <http://www.w3.org/ns/formats/SPARQL_Results_JSON> ,
<<http://www.w3.org/ns/formats/Turtle>> .

sparql 1.1

Posibilități de actualizare a datelor din grafuri RDF:
INSERT DATA, DELETE DATA, MODIFY, LOAD, CLEAR

Operații la nivel de graf RDF:
CREATE, DROP, COPY, MOVE, ADD

specificația curentă: www.w3.org/TR/sparql11-update/
working draft (aug. 2025): www.w3.org/TR/sparql12-update/

sparql 1.1

```
INSERT DATA { # inserăm unele date despre noua eroină :-)  
  :desdemona foaf:title "Dr." ; foaf:name "Dés de Möna" .  
}
```

```
MODIFY <http://castel.info/eveniment/bal-mascot/participanti>  
DELETE { ?persoana foaf:knows :julieta } # substituim :julieta  
INSERT { ?persoana foaf:knows :desdemona } # cu noua venită  
WHERE {  
  ?persoana rdf:type foaf:Person .  
  ?persoana foaf:knows :julieta .  
}
```



fiecare operație
e considerată
atomică

Alte exemple, exerciții și instrumente software:

SPARQL Playground

github.com/calipho-sib/sparql-playground

SPARQL with Wikidata

en.wikibooks.org/wiki/SPARQL

Triply Academy

triplydb.com/academy/

RDF & SPARQL Playground

rdfplayground.dcc.uchile.cl

SPARQL Open Source Projects on Github

awesomeopensource.com/projects/sparql

sparql – privire de ansamblu

SPARQL permite:

extragerea de valori de interes din date (semi-)structurate

explorarea resurselor distribuite via interogări asupra
posibilelor relații ce pot fi stabilite între acestea

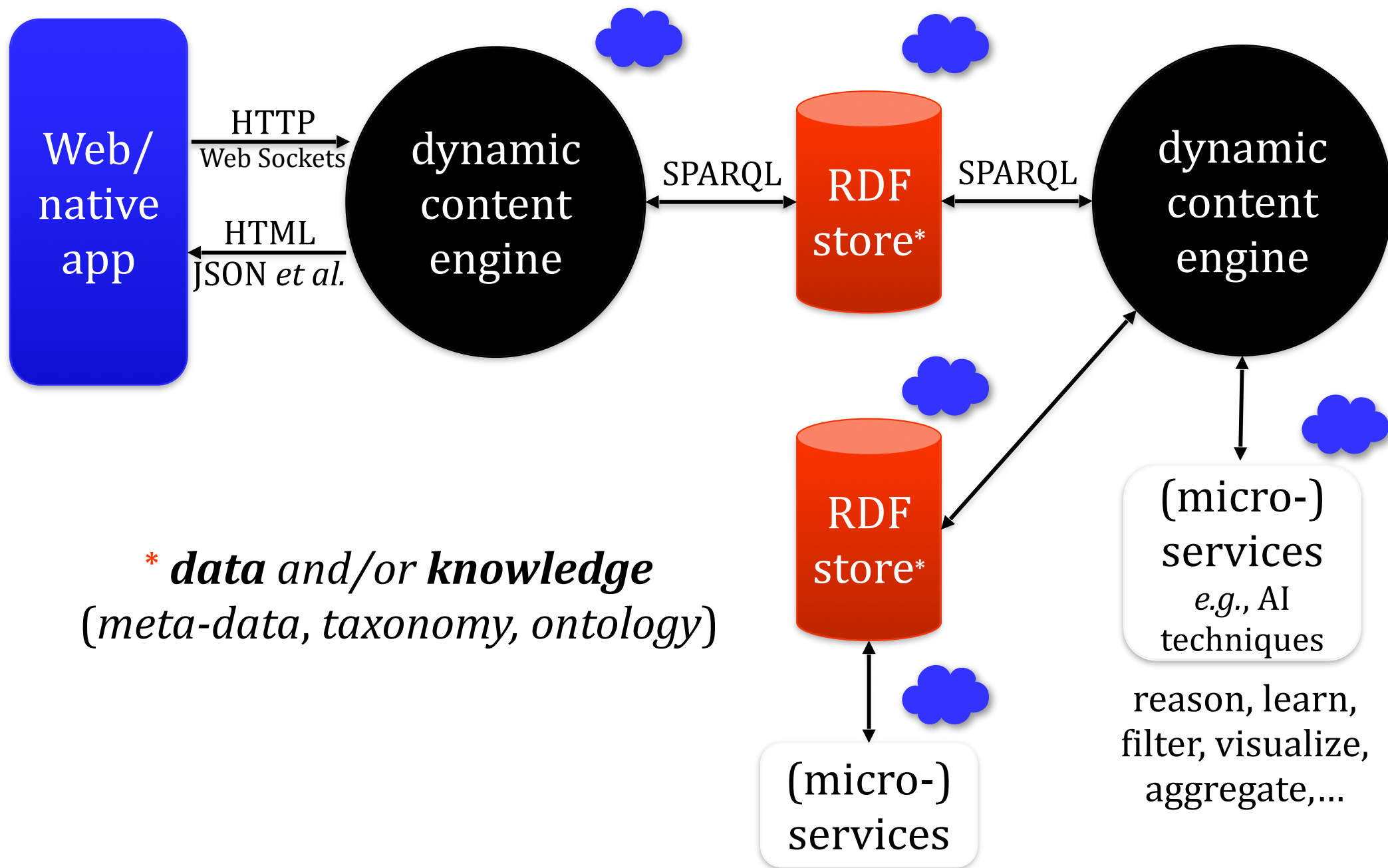
realizarea de *join*-uri complexe

transformarea datelor RDF dintr-un vocabular în altul,
inclusiv cu posibilitatea actualizării

rezumat

obținerea,
transformarea
și interogarea
datelor RDF





episodul viitor:
aplicații Web bazate pe RDF + *Linked Data*